

Kurv

Kurv w Containers

Day 2 - Development

11-Mar-2026

Pythian

Agenda

- Overview
- What is a container
- Best Practices
- Development of Application
- Docker and Docker Compose
- Questions & Next Steps



Goals of the next 3 days

- A fully functional CI/CD pipeline.
 - Application being built from a repo
- A sample application deployed to a cloud environment.
 - Application once built will be deployed
- A roadmap for implementing a modern SDLC in your organization.
 - What could that roadmap look like to you
- All workshop materials, including presentations and code samples.
 - Repo shared

What is a container?

Docker: <https://www.docker.com/resources/what-container/>

A container is a standard unit of software that packages up code and all its dependencies so the application runs quickly and reliably from one computing environment to another.

Microsoft: <https://azure.microsoft.com/en-us/resources/cloud-computing-dictionary/what-is-a-container>

A container, or software container, is a standalone package of software that bundles together application code with the operating system libraries and dependencies required to run it. It can consistently run in any computing environment—whether on a developer's laptop, a test server, or a production cloud service.

Redhat: <https://www.redhat.com/en/topics/containers>

Containers are a technology that allow applications to be packaged and isolated with their entire runtime environment. This makes it easier to maintain consistent behavior and functionality while moving the contained application between environments (dev, test, production) and across public, private, hybrid cloud, and on-premise.

What is a container?

Isn't that the Same as a VM?

Virtual Machine

Operating System - Linux / Windows

Disk Storage - attached storage

Compute - vCPU

Network - TCP

Container

Operating System - Linux / Windows

Disk Storage - attached storage

Compute - vCPU

Network - TCP

What is a container?

Theory vs Practice

Virtual Machine

Operating System - Linux / Windows

Disk Storage - Attached storage

Compute - vCPU (think cores)

Network - TCP (first class)

Long running, OS driven, stable, multiuse, state, many cpu, first class network, the host can be invisible to the VM)

Container

Operating System - Linux / ~~Windows~~

Disk Storage - ~~Attached storage~~

Compute - vCPU (think microseconds)

Network - TCP (NAT only)

Long or short running, Application driven, restart at any time, single use, stateless, micro cpu, separate network (always NAT), the host is the ultimate controller)

What is a container?

At a practical level what does this mean?

Container (Could you, but should you?)

It is an OS that acts like the main application thread.

Single Thread PID 1 - This is your life line, if it fails the container stops running (as a side note you generally cannot kill this thread from inside the container)

Image Matters - Do not install items during boot, the image should have "everything it needs" (Statelessness)

Shutdown - Resiliency and Shutdown speed/gracefulness

OS - Linux only

Disk - Don't bother other than strictly temp files (resilience), state needs to be stored somewhere else. Buckets, Databases, Memstores, Queues (Rabbit or Kafka)

Compute - will be constrained

Memory - will be constrained

Network - only access other containers in the namespace directly and other other configuration

What is a Container?

So what is it good for/bad for?

Container (Could you, but should you?)

Good Things:

- Break a process down into tiny steps (the faster they can be run the better) < 10s is best.
- Gets all of its data from the input or at least enough information to get the data
 - Browser Request
 - REST or graphql Request
 - Queue Message (Rabbit or Kafka (and the like))
 - Are coded to be "resilient". if a process fails ½ way through another process can pick it up without issue.

Bad Things:

- Attaching to storage directly (this will limit ability of the service to scale and distribute)
- During "boot" requiring validations (Databases are a classic example), fail fast/recover fast
- Long running processes which "cannot fail". Never assume a container will not crash. If it does it need to be able to "restart" or ideally "pick up where it left off".

The most basic container

Hello World

We can run Several Images/Containers that are prebuilt in DockerHub

What is DockerHub or more importantly what is Docker?

- Docker - The application or Command
- DockerFile - The file we will use to define the Image we want
- DockerHub - The artifact registry that host most public base images - https://hub.docker.com/_/hello-world
- Docker Shim - Legacy docker image/container support (now the standard *containerd*)
- Docker Compose - A docker host that makes Docker (Commands above) simpler

First up just to see if we can run a container:

```
docker run hello-world
```

Basic Container that does “something”

First we need a Repository

1. Goto Github (<https://github.com/username>)
2. Create “New” repository
 - a. Name: kurv_sdmc
 - b. Visibility: Private
 - c. Readme: On
3. Click “Create Repository”
4. Clone the Repository
 - a. In VSCode ensure you are in “WSL”
 - b. In a terminal window create a folder for the repository (e.g. ~/githubsrc)
 - c. Click the Source Control Icon, paste the “URL” from github and select the folder above
 - d. Open in the same window.
5. Once in the repository
 - a. In a terminal window create a folder for “src/basiccontainer” (i.e. mkdir -p ./src/basiccontainer). The full path of this should be something like “~/githubsrc/kurv_repo/src/basiccontainer”

Basic Container that does “something”

My first container

1. Create DockerFile
2. Add bash script to control output
3. Use an Environment variable to see what to output
4. Move this to a docker compose configuration

Goto: <https://chatgpt.com/>

Prompt: can you create a dockerfile content that uses ubuntu base image and runs a bash script

Follow along and see the basic commands including how to run the docker file.

Prompt: can you create a docker compose file for the above container

Follow along and see the basic commands.

Basic Container that does “something”

My first container

Goto: <https://chatgpt.com/>

Prompt: Can you update the script file to also contain a loop that loops 10000 times and sleeps 1 second per loop also outputting the number on each loop

Update the script file to contain the outputted loop code

Build and run the container

press ctrl+c

What happens, why does it happen?

Basic Container being a good citizen

Shutdown

1. Update the previous script to trap the ctrl+C command
2. Run the container and notice that it will now finish processing directly after the "ctrl+C" command from Docker is sent.

Prompt: what is the code to trap the ctrl+c keystroke to gracefully stop processing

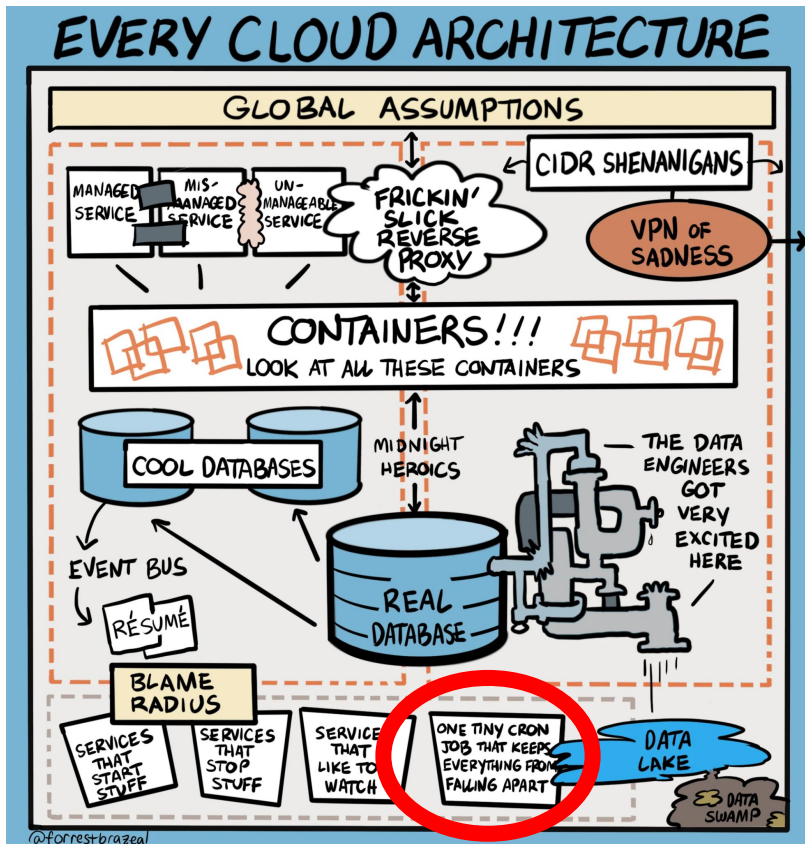
Update the script and rebuild the container.

Docker Compose Commands that you will use a lot:

- `docker-compose build`
- `docker-compose build --no-cache`
- `docker-compose up`
- `docker-compose up --build`
- `docker-compose down`

Basic Container we just built this guy

Every Cloud Architecture



@forrestbrazeal

Create a PR

What is a PR and what is the Action for

Commit your code to your branch.

Push the branch to the repo and then go to the repo.

Create the PR for Review

- show the .github action
- Complete for Today

A Real App

What is our app going to look like?

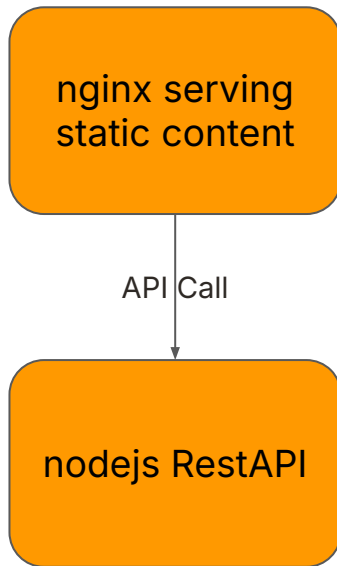
A Basic Web Page with an API Call

We will keep this simple, but it will start to show you how to use a container to host an application with much of the basics in place.

We will have an HTML page that is effectively serving up static content and an API application that is responding with information for the static page.

Create a new folder at the root called "basicapp" and a sub folder called "web". (i.e.

~/githubsrc/kurv_repo/src/basicapp/web)



Create a Static Website

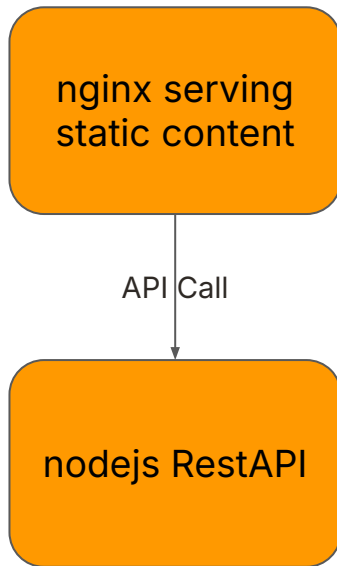
A Basic Web Page with an API Call

Goto: <https://chatgpt.com/>

Prompt: Can you create an html page that is basically a hello world page but looks good.

and you may also need a second prompt: can you separate the html and css files?

Can you now create a dockerfile and a docker-compose file for this container using nginx such that you can see the web pages.



Create a Static Website

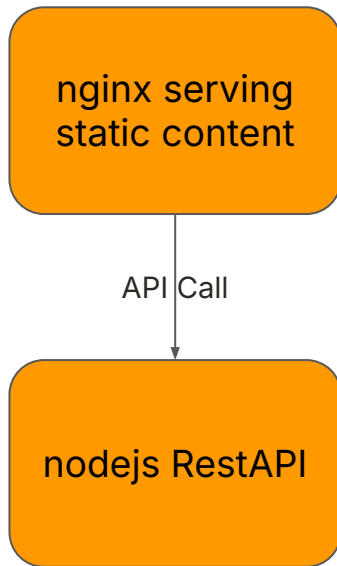
A Basic Web Page with an API Call

Goto: <https://chatgpt.com/>

Can you now create a dockerfile and a docker-compose file for this container using nginx such that you can see the web pages.

Prompt: Can you give me a dockerfile to server the pages via nginx

Prompt: can you setup a docker-compose file to run this container



Create an API Service

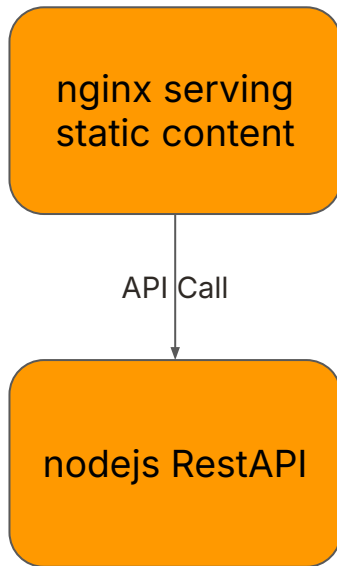
A Basic Web Page with an API Call

Goto: <https://chatgpt.com/>

Now we will need to create a backend API

Prompt: Can you create a second service using nodejs and express to add a hits counter with a local variable as the source of the counter data.

Note: update your files (docker, docker compose) across the board. Use the AI to help you.



Create an API Service

A Basic Web Page with an API Call

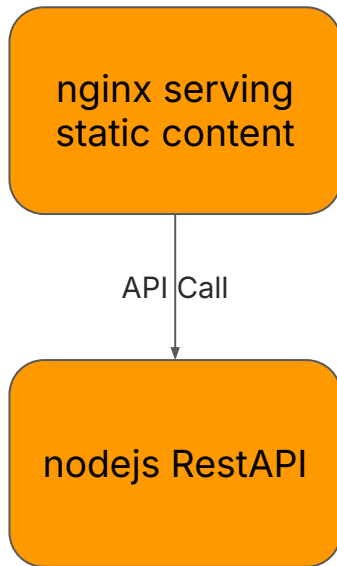
Goto: <https://chatgpt.com/>

Note: update your files (docker, docker compose) across the board. Use the AI to help you.

Prompt: Can you put the web files in their own directory called web and the api files in a separate directory called api?

NOTE: The containers are taking a long time to stop or erroring.

prompt: the api container is not stopping properly when docker compose stops it. can you update the code to fix this.



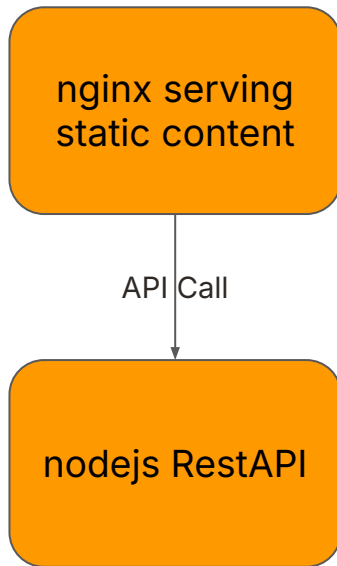
Update the Web Page to display

A Basic Web Page with an API Call

Goto: <https://chatgpt.com/>

Display the hit counter on the web page.

Prompt: Can you update the html page to display the value from the api service.



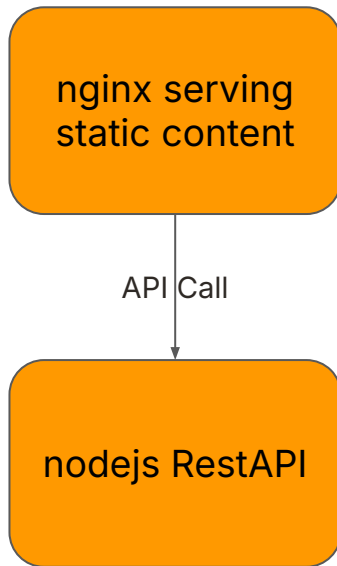
Add Unit Tests (Optional)

A Basic Web Page with an API Call

Goto: <https://chatgpt.com/>

Finally we need to update the API to have unit tests

Prompt: Can you update the api to have unit tests included



Add Environment and Build Arguments

A Basic Web Page with an API Call

Goto: <https://chatgpt.com/>

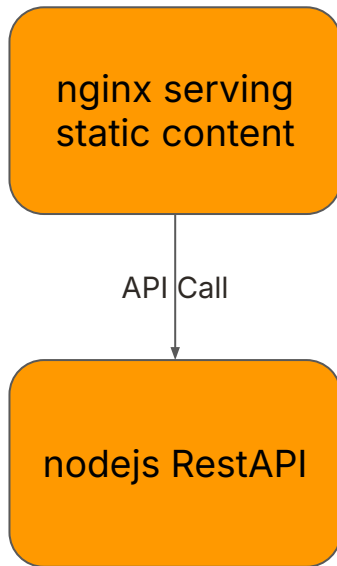
Build Args and Environment Variables:

What needs to change across environments, how do we control it and should it be an environment variable or a build argument (same, but different).

Prompt: add a build arg for the html page to change the url to the api

Prompt: add an environment variable to the api to control what is the starting number of the counter

Note: update your files (docker, docker compose) across the board. Use the AI to help you.



Questions & Next Steps



Thank you